Wyniki etapu III: Projekt

System wspomagania ZPI

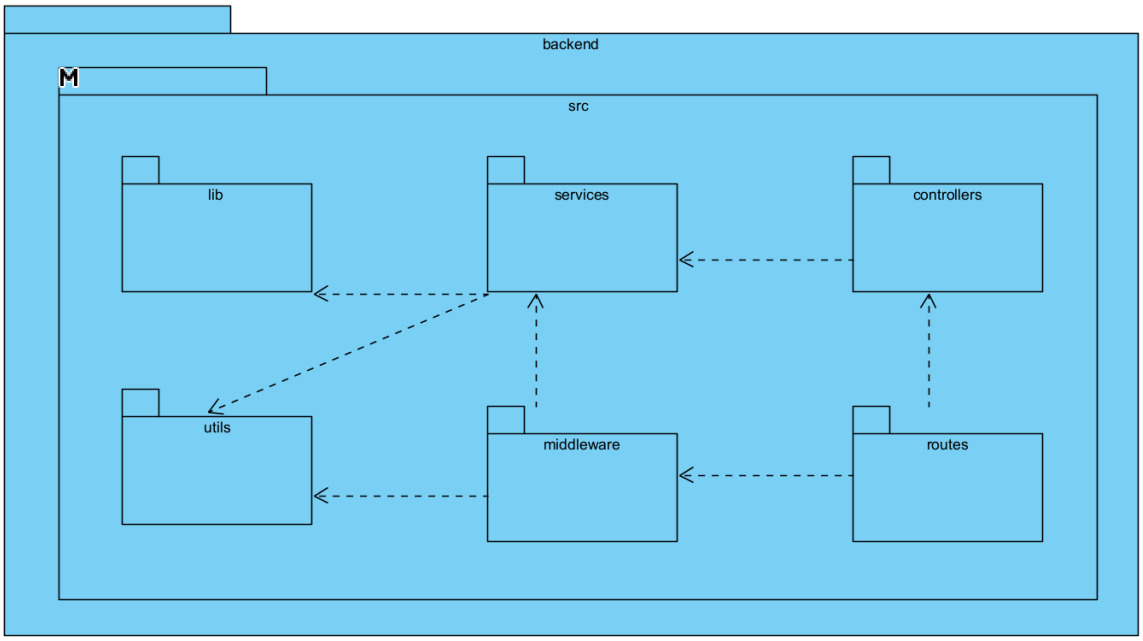
Projektowanie oprogramowania

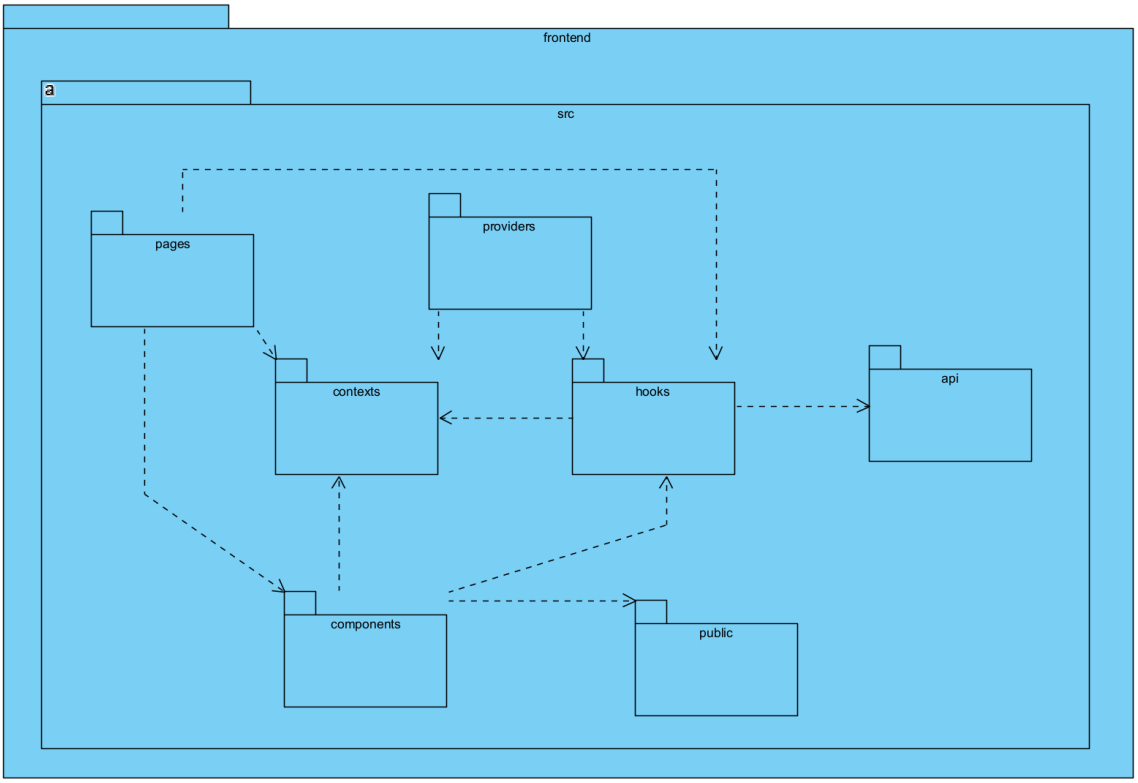
Skład zespołu:
Aleksandra Kozioł (280665),
Maja Kurczyna (280494)

Prowadzący:
dr inż. Prof. Uczelni Bogumiła Hnatkowska

Architektura oprogramowania

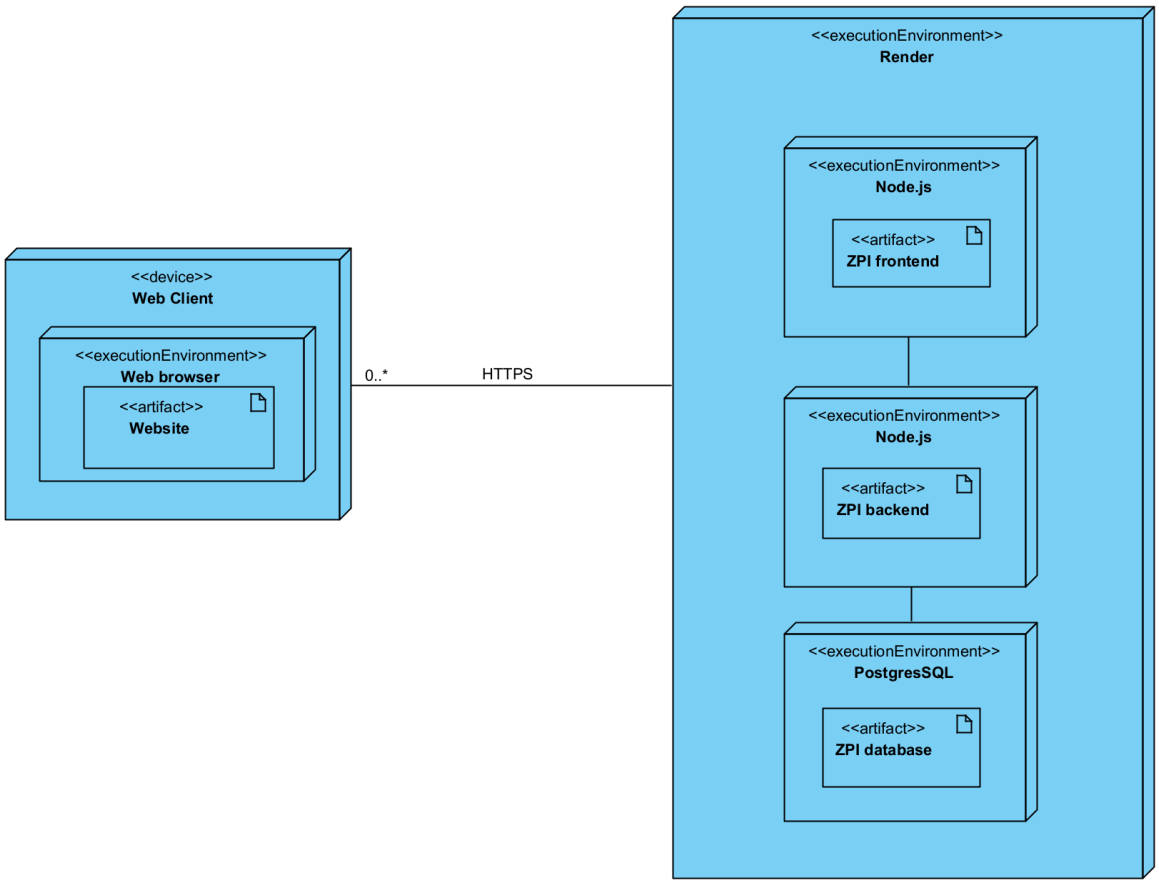
1. Architektura logiczna





- **backend**
  - controllers – zawiera funkcje kontrolerów odpowiedzialne za obsługę wymaganej akcji poprzez wywoływanie funkcji serwisów
  - middleware – zawiera pliki, pozwalające na obsługę błędów oraz weryfikację tokena uwierzytelniającego i wymaganej roli do wykonania akcji
  - routes – zawiera wszystkie zdefiniowane trasy, obsługujące żądania
  - services – zawiera funkcje serwisów, odpowiadające za wykonanie akcji i odpowiednich zapytań do bazy danych
  - utils – zawiera klasy wyjątków, które są rzucane przez funkcje backendu, stworzonych na potrzeby aplikacji w celu lepszej obsługi błędów
  - lib – zawiera singletona klienta bazy danych, za pomocą którego realizowane są zapytania do bazy danych
- **frontend**
  - pages – zawiera pliki .jsx odpowiadające za widok poszczególnych stron w przeglądarce
  - hooks – zawiera pliki z funkcjami, korzystającymi z hooków (funkcji react umożliwiających, m.in. zarządzanie stanem), odpowiadające za obsługę akcji przesyłanych do backendu
  - contexts – zawiera pliki kontekstów, umożliwiających współdzielenie danych, takich jak powiadomienia i okienka
  - components – zawiera wszystkie komponenty wykorzystywane na stronach, np. przyciski akcji w temacie, pasek nawigacyjny
  - api – zawiera funkcję obsługujące żądania API
  - providers – zawiera pliki dostarczające okno modalne, okno powiadomień, które są komponentami wysokiego rzędu opakowujących drzewo komponentów aplikacji
  - public – zawiera pliki z niezbędnymi grafikami, np. logiem Politechniki Wrocławskiej

2. Architektura fizyczna



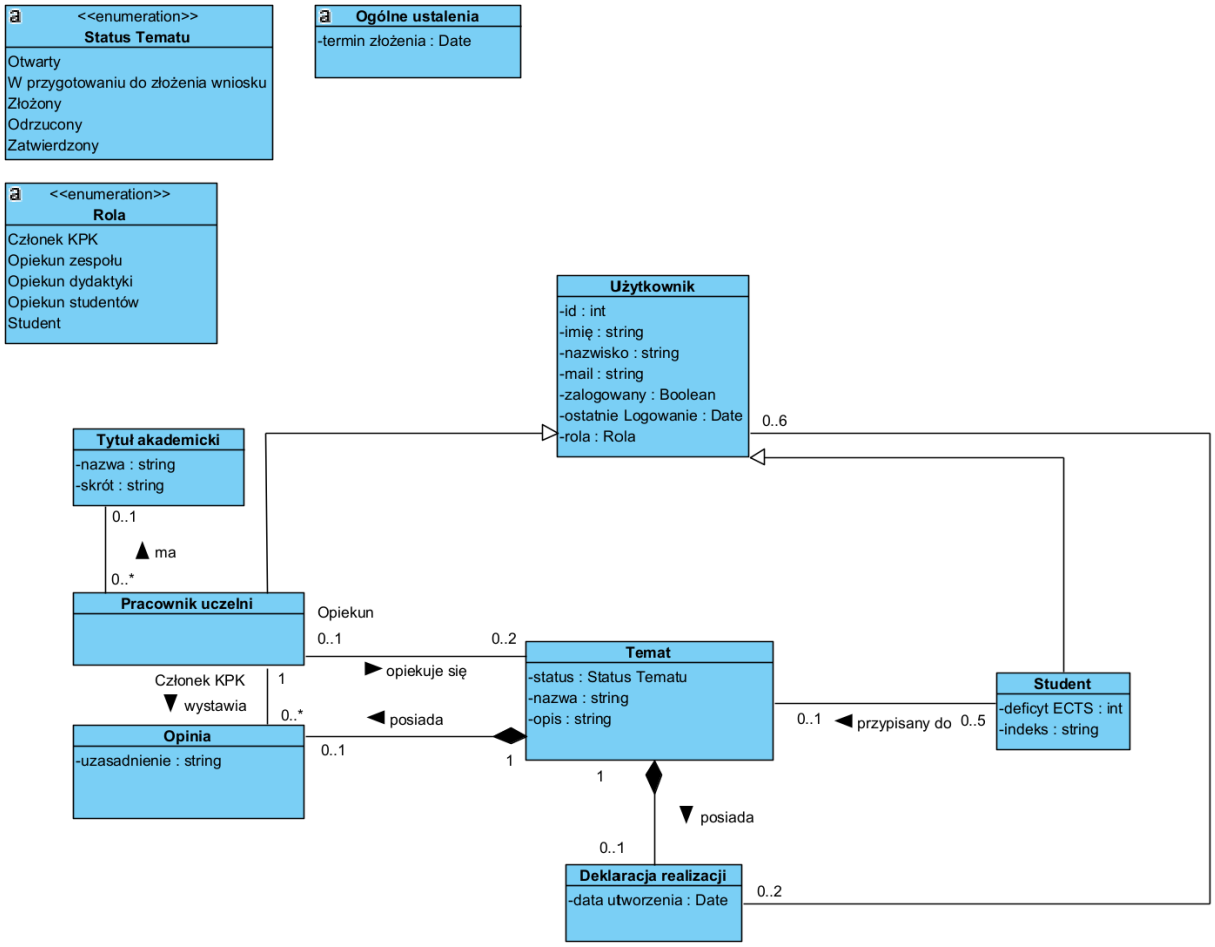
Aplikacja uruchomiona jest w środowisku chmurowym Render. Wykorzystywane są darmowe instancje, bez gwarancji wysokiej dostępności. Komunikacja aplikacji z bazą danych PostgreSQL odbywa się z wykorzystaniem natywnego protokołu PostgreSQL (TCP/5432) w ramach wewnętrznej sieci platformy Render. Komunikacja pomiędzy klientem a frontendem oraz pomiędzy frontendem a backendem realizowana jest za pomocą protokołu HTTPS.

Wersje środowisk uruchomieniowych:

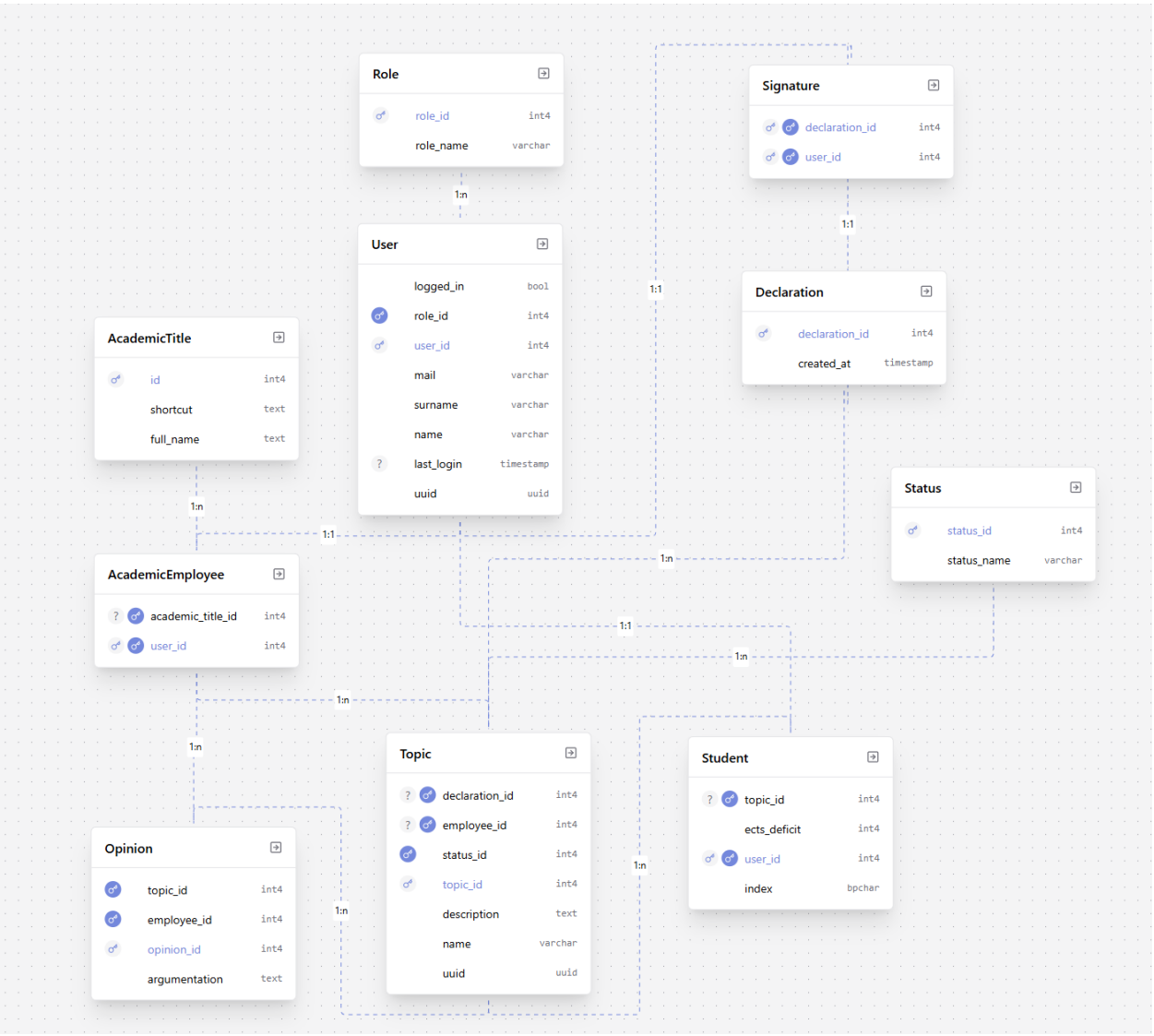
- Node.js (ZPI frontend) w wersji 24.11.1
- Node.js (ZPI backend) w wersji 24.11.1
- PostgreSQL w wersji 18, instancja o parametrach: 256 MB RAM, 0.1 CPU, 1 GB przestrzeni dyskowej.

3. Model danych

Wprowadzono drobne poprawki do modelu informacyjnego. Uzupełniono model o tytuł akademicki oraz dodano atrybut *data utworzenia* do Deklaracji realizacji.

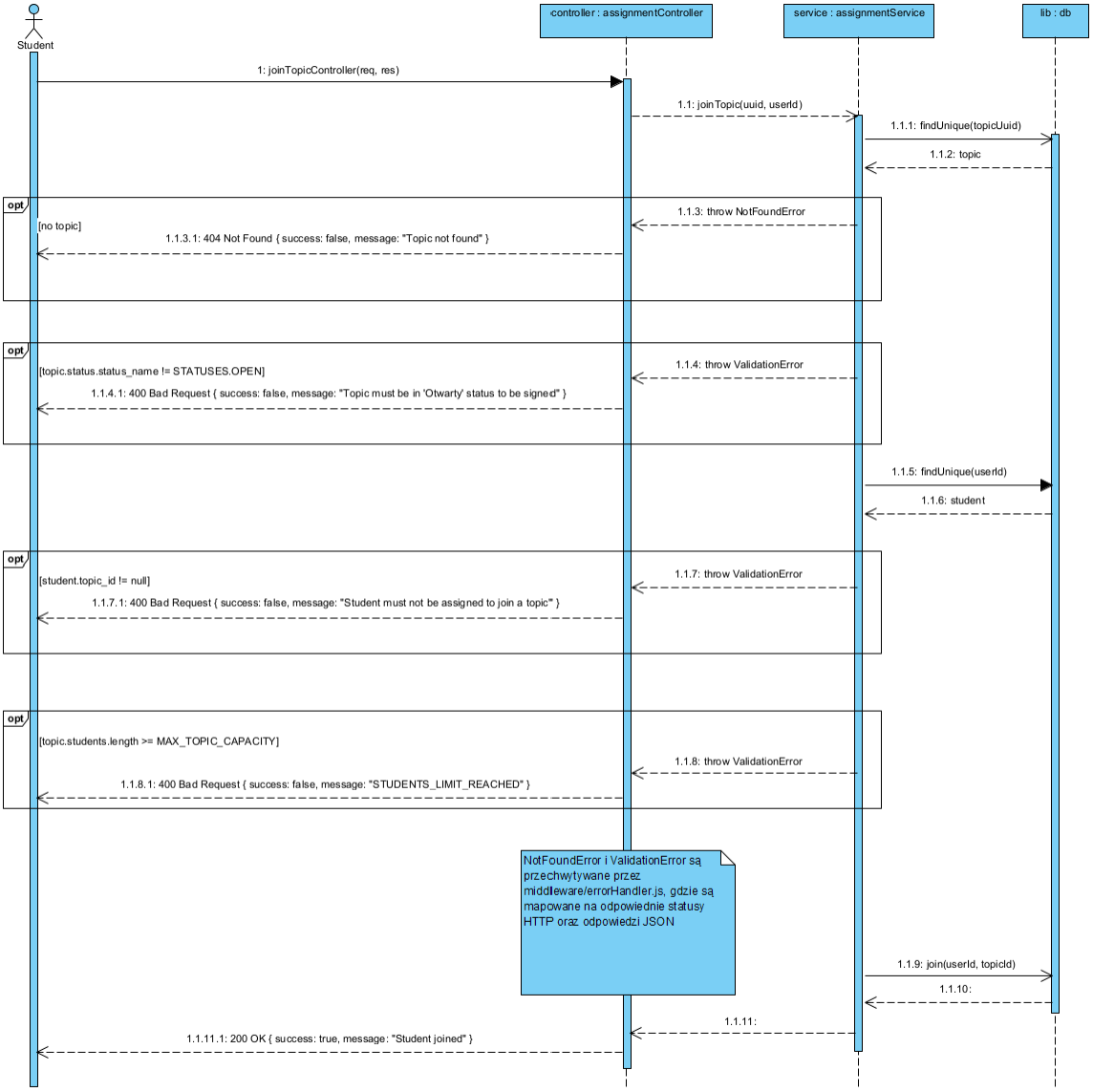


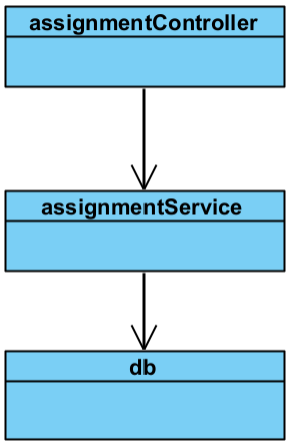
Na podstawie powyższego, poprawionego modelu informacyjnego zdefiniowano model danych.



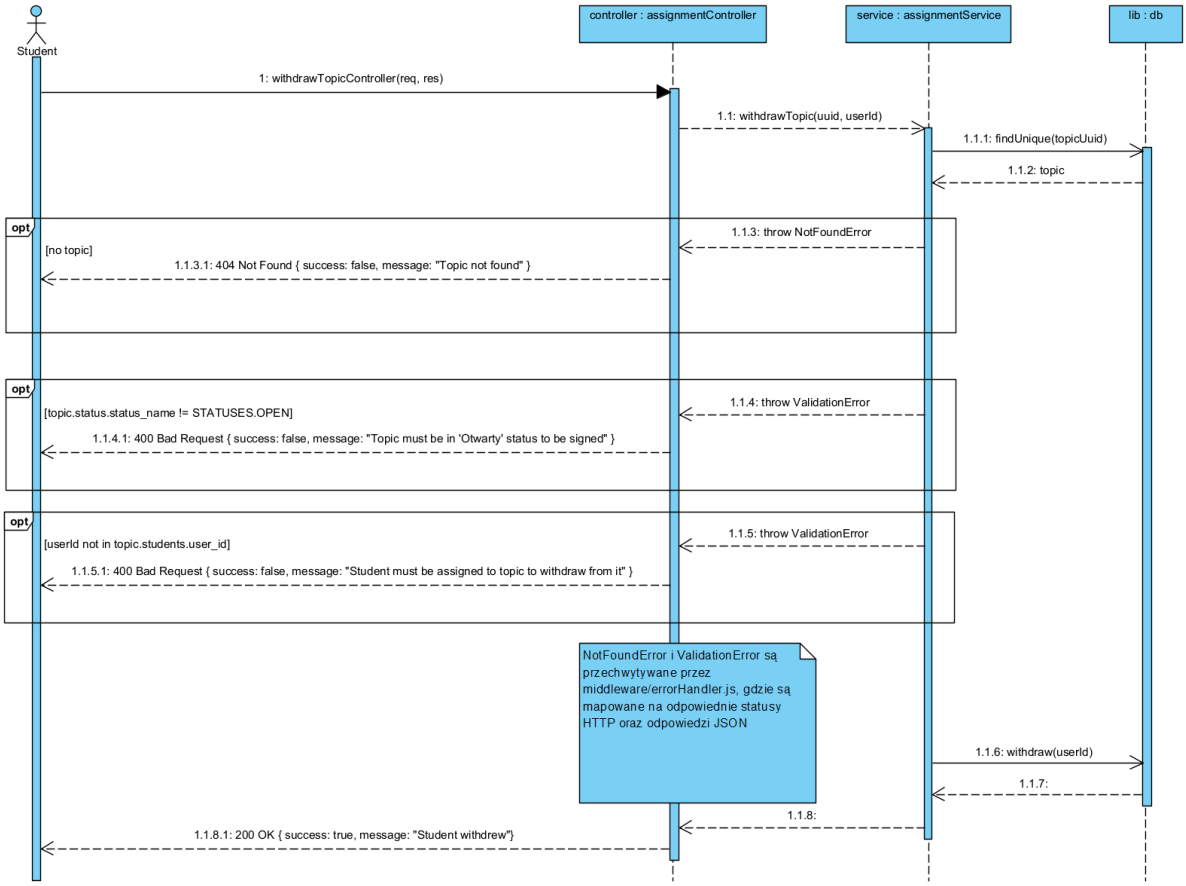
4. Realizacja przypadków użycia

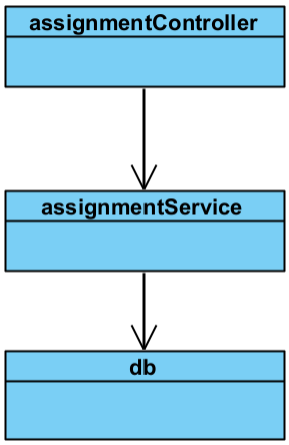
4.1.1 Dopisanie do tematu [Maja Kurczyna]



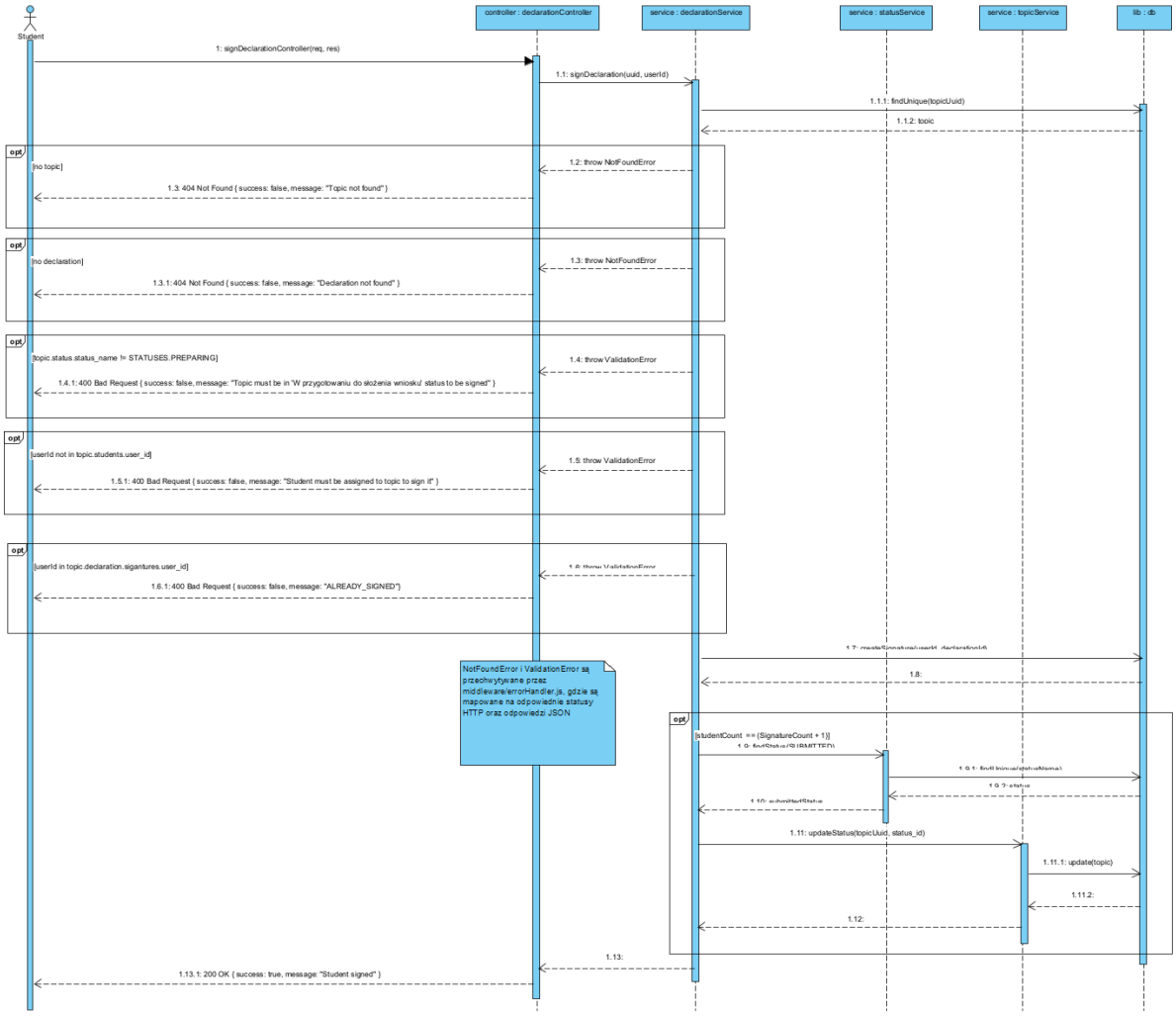


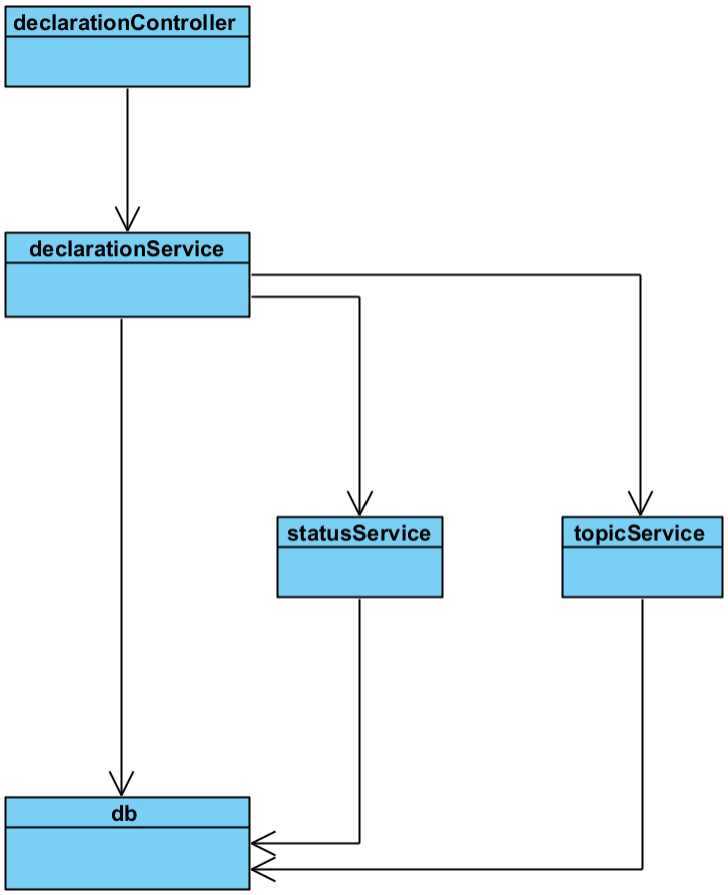
4.1.2 Wypisanie z tematu [Maja Kurczyna]



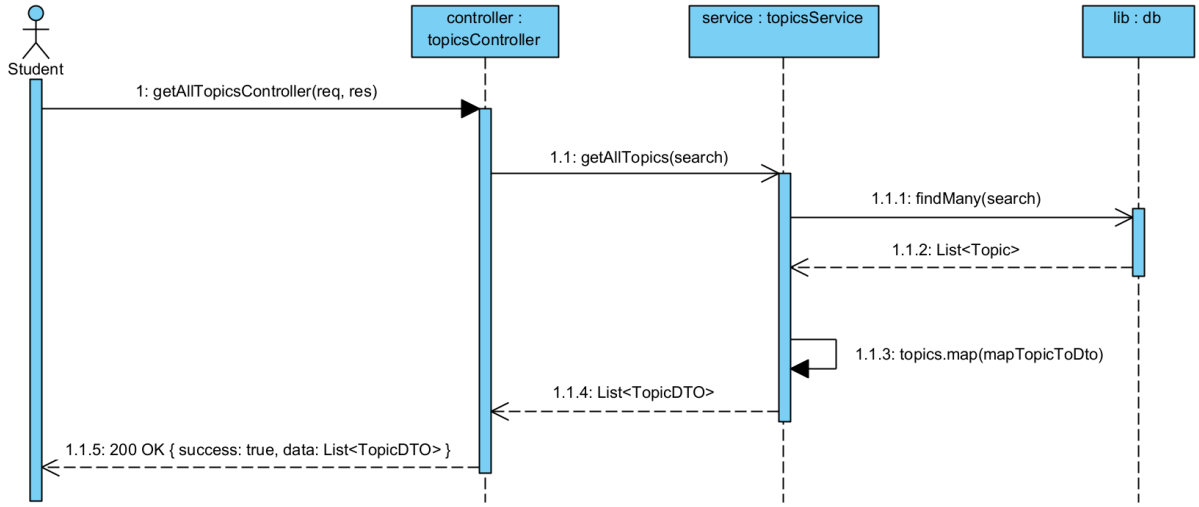


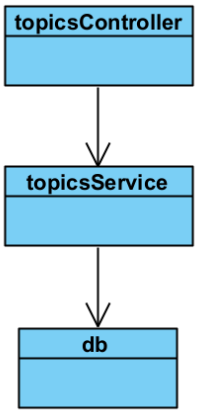
4.1.3 Podpisanie deklaracji [Maja Kurczyna]



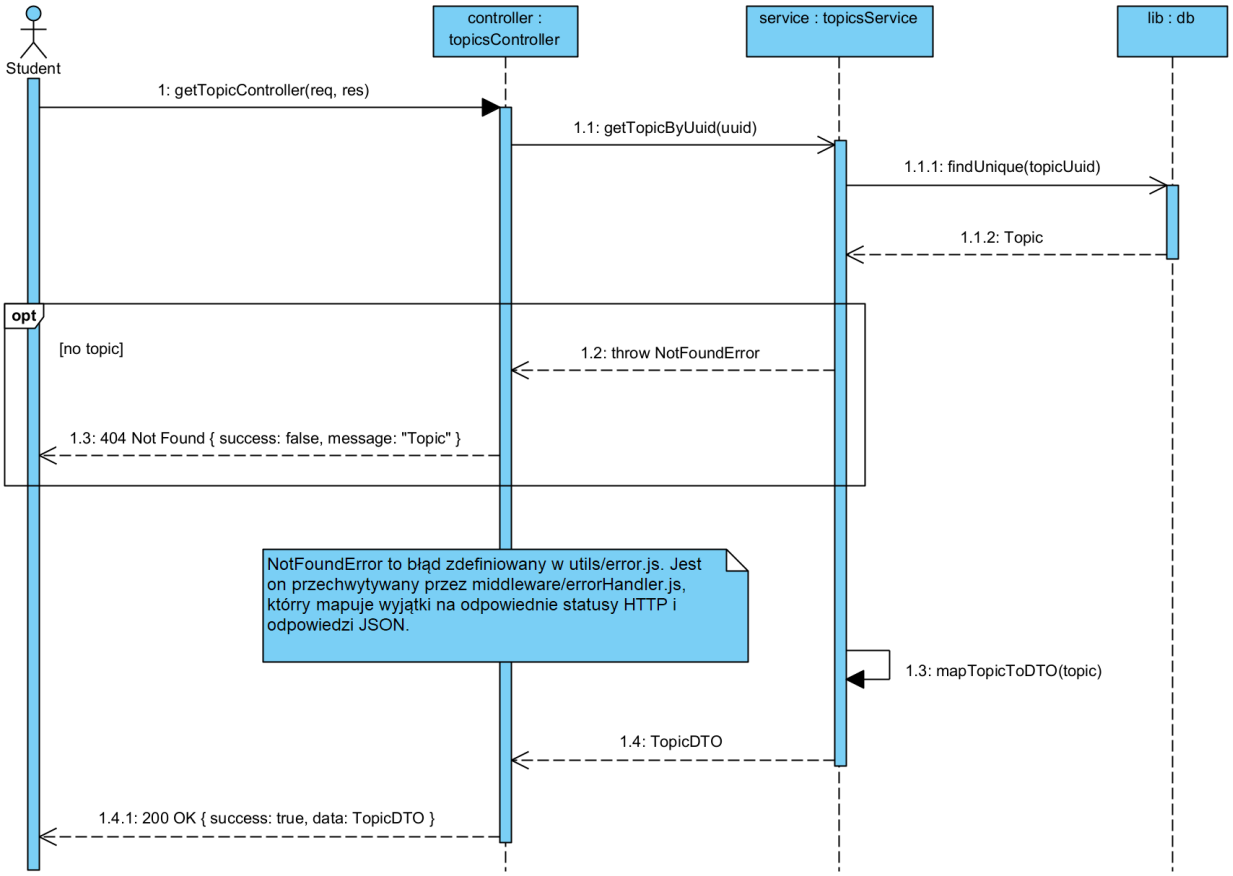


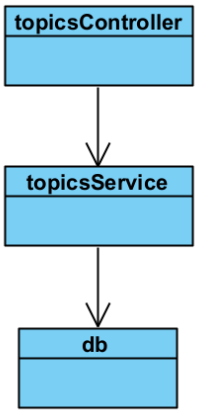
4.1.4 Wyświetlanie tematów przez studenta [Aleksandra Kozioł]





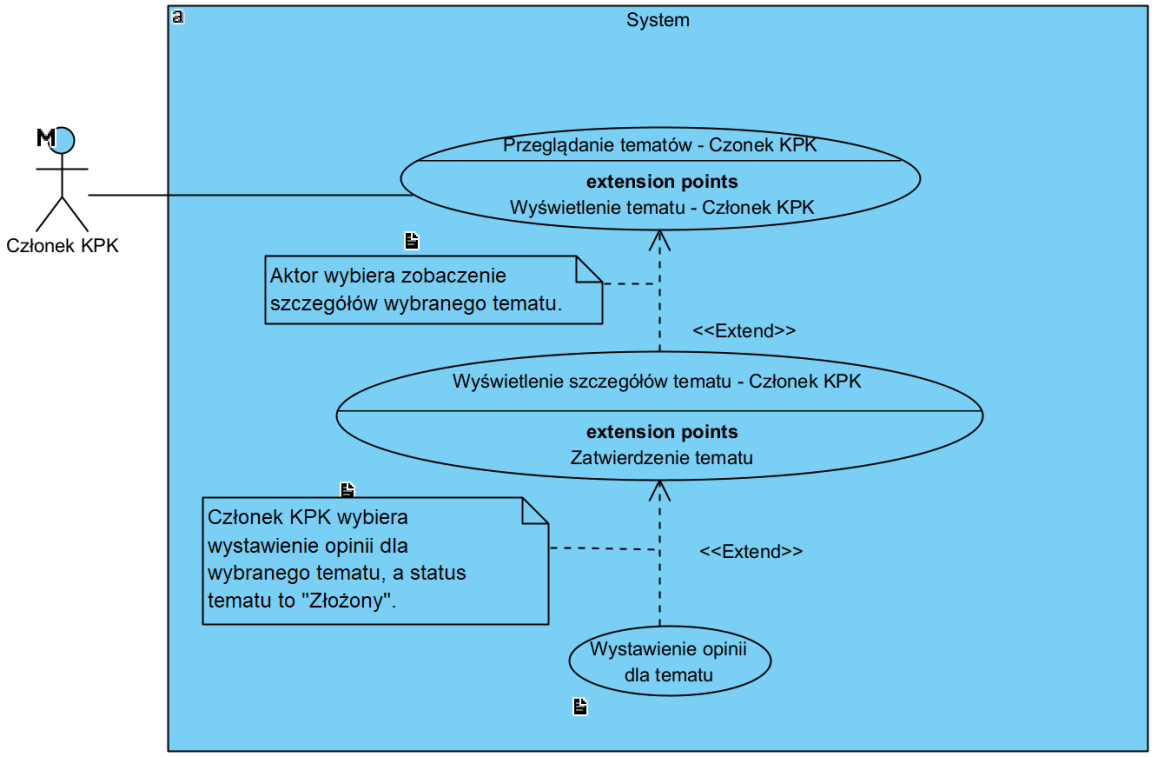
4.1.5 Wyświetlanie tematu przez studenta [Aleksandra Kozioł]

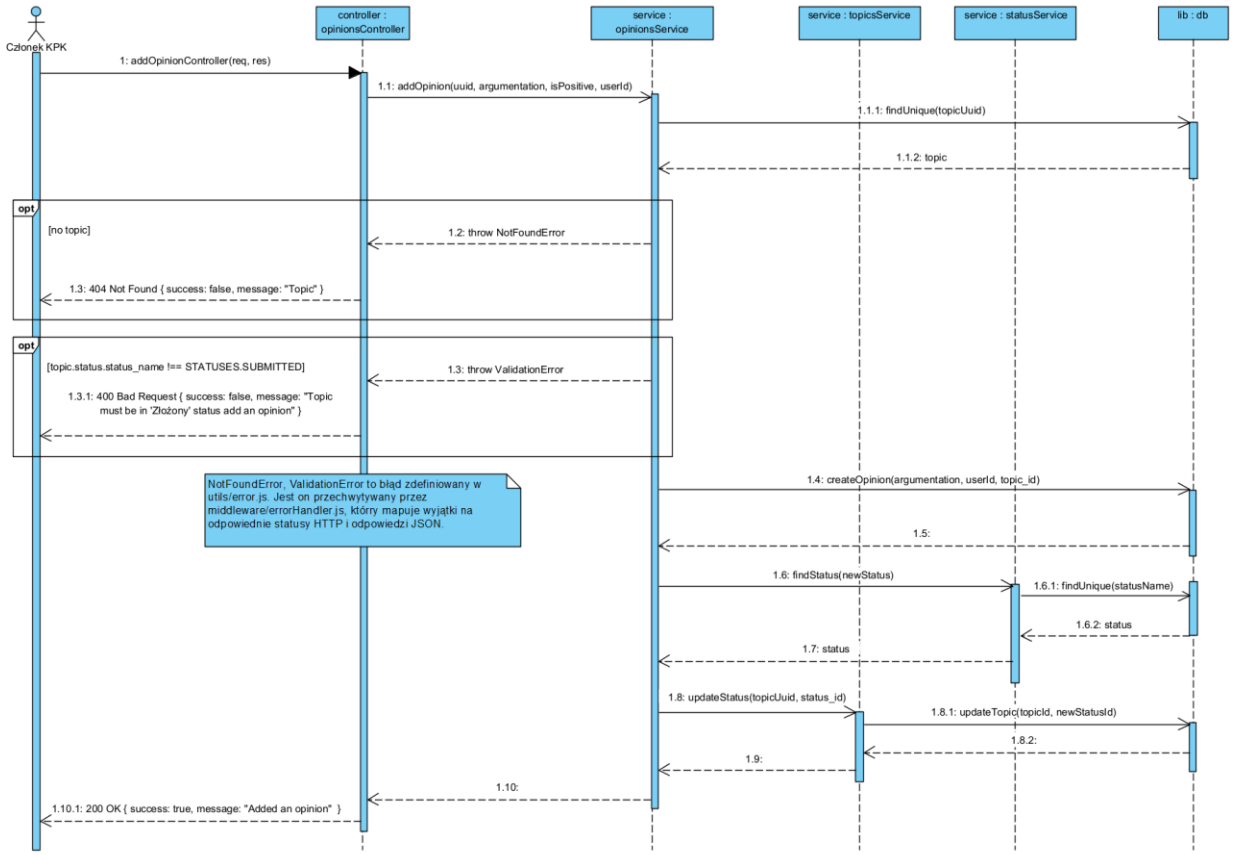


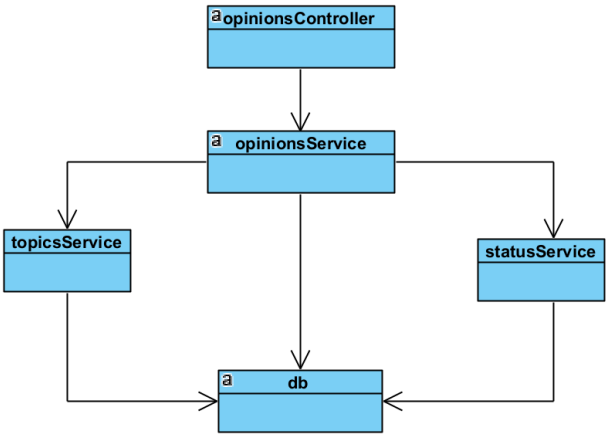


4.1.6 Wystawienie opinii dla tematu przez Członka KPK [Aleksandra Kozioł]

Wystawanie pozytywnej i negatywnej opinii zostało połączone do jednego przypadku użycia po uwagach do ostatniego etapu. W zapytaniu wysyłana jest flaga oznajmiająca czy jest to wystawienie opinii pozytywnej czy negatywnej. Brak weryfikacji rozmiaru opinii w bazie danych jest celowy i ma zapobiegać nakładaniu zbędnych limitów na przechowywane opinie. Walidacja wymagająca co najmniej jednego znaku odbywa się wyłącznie w interfejsie i jest to działanie w pełni zamierzone.







Projekt bazy danych

Prisma – narzędzie do wygenerowania schemata bazy danych
Prisma jest narzędziem ORM dla środowiska Node.js, które pośredniczy w komunikacji pomiędzy aplikacją a bazą danych. Zamiast wykonywać zapytania SQL bezpośrednio, aplikacja korzysta z wygenerowanego klienta Prisma, który udostępnia metody do wykonywania operacji na danych, takich jak tworzenie, odczyt, aktualizacja i usuwanie

rekordów. Prisma składa się z trzech głównych elementów: pliku schemata Prisma *schema.prisma*, Prisma Clienta (w naszym przypadku zdefiniowanego jako singleton w /backend/src/lib/db.js), mechanizmu migracji.

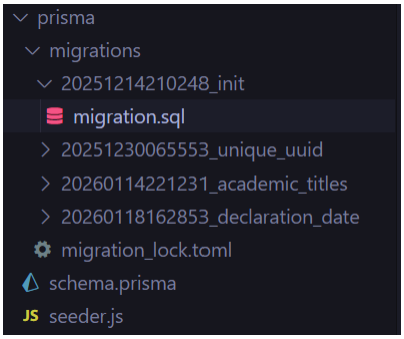
Schemat bazy danych:

```
Generate
generator client {
  provider = "prisma-client-js" // using java script files for compability with node.js
  output   = "../generated/prisma"
}

datasource db {
  provider = "postgresql"
}

model Role {
  role_id   Int    @id @default(autoincrement())
  role_name String @unique @db.VarChar(50)
  users     User[]
}

model Status {
  status_id   Int    @id @default(autoincrement())
  status_name String @unique @db.VarChar(100)
  topics      Topic[]
}

model User {
  user_id    Int      @id @default(autoincrement())
  uuid       String   @db.Uuid @default(uuid()) @unique
  name       String   @db.VarChar(50)
  surname    String   @db.VarChar(50)
  mail       String   @unique @db.VarChar(255)
  last_login DateTime?
  logged_in  Boolean  @default(false)

  role_id Int
  role    Role @relation(fields: [role_id], references: [role_id], onDelete: Restrict)

  academicEmployees AcademicEmployee[]
  students          Student[]
  signatures        Signature[]

  @@index([role_id])
  @@index([uuid])
}

model AcademicTitle {
  id        Int                @id @default(autoincrement())
  full_name String             @unique()
  shortcut  String
  employees AcademicEmployee[]
}

model AcademicEmployee {
  user_id           Int            @id
  user              User           @relation(fields: [user_id], references: [user_id], onDelete: Cascade)
  academic_title_id Int?
  academic_title    AcademicTitle? @relation(fields: [academic_title_id], references: [id])

  opinions          Opinion[]
  topics            Topic[]
}


model Student {
  user_id Int @id
  user    User @relation(fields: [user_id], references: [user_id], onDelete: Cascade)

  index        String @db.Char(6)
  ects_deficit Int

  topic_id Int?
  topic    Topic? @relation(fields: [topic_id], references: [topic_id], onDelete: Restrict)

  @@index([topic_id])
}
```

```
73  model Topic {
74    topic_id  Int    @id @default(autoincrement())
75    uuid      String @db.Uuid @default(uuid()) @unique
76    name      String @db.VarChar(1000)
77    description String
78
79    status_id Int
80    status    Status @relation(fields: [status_id], references: [status_id], onDelete: Restrict)
81
82    employee_id Int?
83    employee    AcademicEmployee? @relation(fields: [employee_id], references: [user_id], onDelete: Restrict)
84
85    declaration_id Int?
86    declaration    Declaration? @relation(fields: [declaration_id], references: [declaration_id], onDelete: Cascade)
87
88    opinion Opinion?
89
90    students Student[]
91
92    @@index([status_id])
93    @@index([employee_id])
94    @@index([declaration_id])
95    @@index([uuid])
96  }
97
98  model Opinion {
99    opinion_id    Int    @id @default(autoincrement())
100   argumentation String
101
102   employee_id Int
103   employee    AcademicEmployee @relation(fields: [employee_id], references: [user_id], onDelete: Restrict)
104
105   topic_id Int @unique
106   topic    Topic @relation(fields: [topic_id], references: [topic_id], onDelete: Cascade)
107
108   @@index([employee_id])
109 }
110
111 model Declaration {
112   declaration_id Int @id @default(autoincrement())
113   created_at DateTime @default(now())
114   topics         Topic[]
115   signatures     Signature[]
116 }
117
118 model Signature {
119   user_id        Int
120   declaration_id Int
121
122   user        User        @relation(fields: [user_id], references: [user_id], onDelete: Cascade)
123   declaration Declaration @relation(fields: [declaration_id], references: [declaration_id], onDelete: Cascade)
124
125   @@id([user_id, declaration_id])
126   @@index([declaration_id])
127 }
```

Powyżej utworzone są modele danych odpowiadające tabelom w bazie danych (wraz z typami, relacjami, ograniczeniami).

Generując migracje Prisma generuje pliki migracyjne, zawierające instrukcje SQL, które są wykonywane na docelowej bazie danych, co prowadzi do utworzenia lub aktualizacji tabel, relacji i indeksów.



Na następnej stronie załączono wygenerowany plik SQL dla migracji *init*.

```sql
-- CreateTable
CREATE TABLE "Role" (
    "role_id" SERIAL NOT NULL,
    "role_name" VARCHAR(50) NOT NULL,

    CONSTRAINT "Role_pkey" PRIMARY KEY ("role_id")
);

-- CreateTable
CREATE TABLE "Status" (
    "status_id" SERIAL NOT NULL,
    "status_name" VARCHAR(100) NOT NULL,

    CONSTRAINT "Status_pkey" PRIMARY KEY ("status_id")
);

-- CreateTable
CREATE TABLE "User" (
    "user_id" SERIAL NOT NULL,
    "uuid" UUID NOT NULL,
    "name" VARCHAR(50) NOT NULL,
    "surname" VARCHAR(50) NOT NULL,
    "mail" VARCHAR(255) NOT NULL,
    "last_login" TIMESTAMP(3),
    "logged_in" BOOLEAN NOT NULL DEFAULT false,
    "role_id" INTEGER NOT NULL,

    CONSTRAINT "User_pkey" PRIMARY KEY ("user_id")
);

-- CreateTable
CREATE TABLE "AcademicEmployee" (
    "user_id" INTEGER NOT NULL,

    CONSTRAINT "AcademicEmployee_pkey" PRIMARY KEY ("user_id")
);

-- CreateTable
CREATE TABLE "Student" (
    "user_id" INTEGER NOT NULL,
    "index" CHAR(6) NOT NULL,
    "ects_deficit" INTEGER NOT NULL,
    "topic_id" INTEGER,

    CONSTRAINT "Student_pkey" PRIMARY KEY ("user_id")
);
```

```sql
-- CreateTable
CREATE TABLE "Topic" (
    "topic_id" SERIAL NOT NULL,
    "uuid" UUID NOT NULL,
    "name" VARCHAR(1000) NOT NULL,
    "description" TEXT NOT NULL,
    "status_id" INTEGER NOT NULL,
    "employee_id" INTEGER,
    "declaration_id" INTEGER,

    CONSTRAINT "Topic_pkey" PRIMARY KEY ("topic_id")
);

-- CreateTable
CREATE TABLE "Opinion" (
    "opinion_id" SERIAL NOT NULL,
    "argumentation" TEXT NOT NULL,
    "employee_id" INTEGER NOT NULL,
    "topic_id" INTEGER NOT NULL,

    CONSTRAINT "Opinion_pkey" PRIMARY KEY ("opinion_id")
);

-- CreateTable
CREATE TABLE "Declaration" (
    "declaration_id" SERIAL NOT NULL,

    CONSTRAINT "Declaration_pkey" PRIMARY KEY ("declaration_id")
);

-- CreateTable
CREATE TABLE "Signature" (
    "user_id" INTEGER NOT NULL,
    "declaration_id" INTEGER NOT NULL,

    CONSTRAINT "Signature_pkey" PRIMARY KEY ("user_id","declaration_id")
);
```

```sql
-- CreateIndex
CREATE UNIQUE INDEX "Role_role_name_key" ON "Role"("role_name");

-- CreateIndex
CREATE UNIQUE INDEX "Status_status_name_key" ON "Status"("status_name");

-- CreateIndex
CREATE UNIQUE INDEX "User_mail_key" ON "User"("mail");

-- CreateIndex
CREATE INDEX "User_role_id_idx" ON "User"("role_id");

-- CreateIndex
CREATE INDEX "Student_topic_id_idx" ON "Student"("topic_id");

-- CreateIndex
CREATE INDEX "Topic_status_id_idx" ON "Topic"("status_id");

-- CreateIndex
CREATE INDEX "Topic_employee_id_idx" ON "Topic"("employee_id");

-- CreateIndex
CREATE INDEX "Topic_declaration_id_idx" ON "Topic"("declaration_id");

-- CreateIndex
CREATE UNIQUE INDEX "Opinion_topic_id_key" ON "Opinion"("topic_id");

-- CreateIndex
CREATE INDEX "Opinion_employee_id_idx" ON "Opinion"("employee_id");

-- CreateIndex
CREATE INDEX "Signature_declaration_id_idx" ON "Signature"("declaration_id");

-- AddForeignKey
ALTER TABLE "User" ADD CONSTRAINT "User_role_id_fkey" FOREIGN KEY ("role_id") REFERENCES "Role"("role_id") ON DELETE RESTRICT ON UPDATE CASCADE;

-- AddForeignKey
ALTER TABLE "AcademicEmployee" ADD CONSTRAINT "AcademicEmployee_user_id_fkey" FOREIGN KEY ("user_id") REFERENCES "User"("user_id") ON DELETE CASCADE ON UPDATE CASCADE;

-- AddForeignKey
ALTER TABLE "Student" ADD CONSTRAINT "Student_user_id_fkey" FOREIGN KEY ("user_id") REFERENCES "User"("user_id") ON DELETE CASCADE ON UPDATE CASCADE;

-- AddForeignKey
ALTER TABLE "Student" ADD CONSTRAINT "Student_topic_id_fkey" FOREIGN KEY ("topic_id") REFERENCES "Topic"("topic_id") ON DELETE RESTRICT ON UPDATE CASCADE;

-- AddForeignKey
ALTER TABLE "Topic" ADD CONSTRAINT "Topic_status_id_fkey" FOREIGN KEY ("status_id") REFERENCES "Status"("status_id") ON DELETE RESTRICT ON UPDATE CASCADE;

-- AddForeignKey
ALTER TABLE "Topic" ADD CONSTRAINT "Topic_employee_id_fkey" FOREIGN KEY ("employee_id") REFERENCES "AcademicEmployee"("user_id") ON DELETE RESTRICT ON UPDATE CASCADE;

-- AddForeignKey
ALTER TABLE "Topic" ADD CONSTRAINT "Topic_declaration_id_fkey" FOREIGN KEY ("declaration_id") REFERENCES "Declaration"("declaration_id") ON DELETE CASCADE ON UPDATE CASCADE;

-- AddForeignKey
ALTER TABLE "Opinion" ADD CONSTRAINT "Opinion_employee_id_fkey" FOREIGN KEY ("employee_id") REFERENCES "AcademicEmployee"("user_id") ON DELETE RESTRICT ON UPDATE CASCADE;

-- AddForeignKey
ALTER TABLE "Opinion" ADD CONSTRAINT "Opinion_topic_id_fkey" FOREIGN KEY ("topic_id") REFERENCES "Topic"("topic_id") ON DELETE CASCADE ON UPDATE CASCADE;

-- AddForeignKey
ALTER TABLE "Signature" ADD CONSTRAINT "Signature_user_id_fkey" FOREIGN KEY ("user_id") REFERENCES "User"("user_id") ON DELETE CASCADE ON UPDATE CASCADE;

-- AddForeignKey
ALTER TABLE "Signature" ADD CONSTRAINT "Signature_declaration_id_fkey" FOREIGN KEY ("declaration_id") REFERENCES "Declaration"("declaration_id") ON DELETE CASCADE ON UPDATE CASCADE;
```